Journal of Intelligent Manufacturing
https://doi.org/10.1007/s10845-025-02672-8

Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach

Darío G. Lema[1] · Lidia Sánchez-González[2] · Rubén Usamentiaga[1] · F. J. delaCalle[1]

Received: 13 June 2025 / Accepted: 6 August 2025
© The Author(s) 2025

Abstract
Automated surface defect detection has been a key research topic for many years, with deep learning-based object detection being one of the most widely used approaches. However, comparing the results of different models remains a challenge due to the use of varying dataset partitions and the stochastic nature of training, which can introduce variability in outcomes. This study highlights that improvements in performance metrics, such as average precision (AP_{50}), do not always reflect a model's true effectiveness, as other factors may influence these results. To address this challenge, a robust methodology is proposed, specifically designed for small datasets, which utilizes analysis of variance and Tukey's test to ensure statistical significance. This methodology provides a reliable and reproducible framework for comparing results across models. The proposed methodology is demonstrated using the latest object detection models and the Northeastern University surface defect dataset, revealing that recent advancements do not always lead to statistically significant improvements. The source code has been made publicly available to promote reproducibility.

Keywords Performance evaluation · Model comparison · Object detection methodology · Statistical tests · Surface defect detection

Introduction

The detection of surface defects in industrial processes is essential for ensuring the quality and reliability of products. Early and accurate identification of potential imperfections is crucial for maintaining production standards and reducing costs associated with defective products.

There are various types of defects, but surface defects, such as patches or scratches, are particularly critical, as they not only compromise the appearance of the final product but also can affect its structural integrity and mechanical functionality. If these surface defects are not detected and corrected in time, they can weaken the component and lead to performance failures, increasing the risk of breakdowns and resulting in additional costs (Colosimo et al., 2020).

Traditionally, the inspection of components has been carried out manually, with an operator visually examining each part to identify potential defects (Usamentiaga et al., 2017). This approach presents two main drawbacks: the subjectivity of the inspection process and the inability to inspect all pieces within a reasonable time frame. Currently, the approach has evolved towards the development of automated methods that enable the objective and precise detection of defects in very short times (delaCalle et al., 2025; Ding et al., 2024; Lema et al., 2023, 2024). Various methods exist, such as the use of magnetic particles, where a magnetic field is applied to the component, causing the particles to adhere to areas with surface defects, thereby revealing their presence (Zolfaghari et al., 2018). Another common method is the use of penetrant liquids, which, when applied to the surface, infiltrate defects and subsequently allow for clear visualization of imperfections by emerging residual liquid through a developer film (Lobanova et al., 2021).

Although these automated methods are commonly used, they have several drawbacks, such as the high cost of the materials required and the extended time needed to perform

✉ Darío G. Lema
gonzalezdario@uniovi.es

[1] Department of Computer Science and Engineering, University of Oviedo, Campus de Viesques, 33204 Gijón, Asturias, Spain

[2] Department of Mechanical, Computer Science and Aerospace Engineering, Universidad de León, 24071 León, León, Spain

Published online: 06 September 2025

a complete inspection. For this reason, there has been a growing interest in computer vision techniques, which offer a faster and more cost-effective solution. These techniques require only cameras, a specialized algorithm, and the appropriate hardware to run it.

In computer vision, traditional methods require a manual evaluation of the characteristics of the defects to be identified, which involves designing specific algorithms for each type of defect. However, recent advances in deep learning have revolutionized this field, enabling neural networks to automatically learn to identify the most relevant features of defects without the need for explicit programming. This has led to an increase in the research, with most current articles focusing on applying deep learning to enhance the accuracy and efficiency of surface defect detection.

However, despite the significant improvements in results brought about by deep learning, defect detection systems still face several challenges:

- High variability in sizes: the sizes of the same type of defect can vary considerably between different pieces. Moreover, there are significant differences in dimensions and aspect ratios among various types of defects. As detailed in "Materials and methods" section, there are instances where a defect may occupy only a few pixels, while in other cases, it can cover a much larger area. Additionally, defects can take on different shapes: some are square, others rectangular, and some have irregular forms. This variability notably complicates the precise detection of defects.
- Inconsistent shapes: unlike other types of objects, defects in steel strips exhibit a notable lack of uniformity. Sometimes, areas marked as defects include sections that are not actually defects. This inconsistency affects the quality of the labeling in the dataset, thus limiting the learning capacity of object detectors.
- Limited number of examples: certain types of defects have a low probability of occurrence, yet when they do occur, they can cause significant issues in production processes. This poses a common challenge in the development of deep learning-based solutions, as building effective models requires a substantial amount of accurately labeled examples. However, labeling large datasets is a time-consuming process. In scenarios such as surface defect detection, this challenge is further compounded by the inherent uncertainty in identifying and accurately labeling defects, which restricts the learning and effectiveness of object detectors.

Over the past decade, various models have been developed specifically to improve the performance of surface defect detection, addressing the limitations of general-purpose models. However, many of these approaches achieve only marginal improvements compared to the baseline implementations they build upon. Additionally, sharing source code to enable reproducibility of results is not a widespread practice. While proposed architectures are described, critical implementation details are frequently omitted, making it difficult to replicate the reported outcomes.

Another significant challenge is the lack of consistency in dataset partitioning (Xie et al., 2024). Even when multiple studies use the same surface defect dataset, it is common for researchers to employ different partitions for training and testing, complicating direct and accurate comparisons of the reported results.

To address these limitations, this work proposes the following contributions:

- Proposal of a robust and consistent methodology: A new strategy is presented for evaluating the performance of different models on small datasets, especially in scenarios with high intrinsic variability in the objects to be detected. This methodology enables reliable comparisons between models, facilitating the analysis of their performance in real-world industrial contexts.
- Reduction of bias in data partitioning: A stratified partitioning approach is introduced, dividing the dataset into four equally sized partitions, ensuring that each partition is used both for training and testing. This allows the capture of inherent variability in defects and reduces bias associated with arbitrary partition selection, which is common in previous studies.
- Rigorous statistical analysis of results: To ensure the validity of comparisons between models, the results are analyzed using ANOVA and Tukey's test, allowing for the precise and reliable identification of significant differences between the evaluated models. This ensures that advances in defect detection are measured in a statistically significant way.
- Reproducible evaluation framework: Through this proposal, the reproducibility of results in future research is encouraged. By providing a clear and detailed methodology, a solid foundation is offered for other researchers to replicate the experiments and validate the results obtained.

In "Literature review" section reviews studies focused on surface defect detection in metal strips using small datasets. Based on the analysis of the reported results, it is concluded that a standardized methodology is needed to ensure consistent evaluation in such scenarios. Without a common framework, it becomes difficult to compare findings across studies, which compromises the reproducibility of results. To address this issue, the proposed methodology

is introduced in "Methodological proposal for model comparison: cross-validation and statistical analysis" section. In "Materials and methods" section presents the dataset used to validate the methodology, along with the evaluation metrics. In "Defect detection models to be evaluated" section describes the object detection algorithms implemented based on prior works. In "Experimental setup" section summarizes the evaluated detection models, the dataset, and the experimental setup. The results are discussed in "Results and discussion" section, with emphasis on their statistical significance. Finally, in "Conclusion" section offers a constructive critique aimed at improving the reproducibility and comparability of future studies.

Literature review

In recent years, significant advances have been made in surface defect detection using deep learning algorithms. These advances rely on the availability of large volumes of data representative of the problem to be solved (Lin et al., 2025). In the case of surface defect detection, it is essential to have images that include proper annotation of the defects present. This need has limited the number of public datasets available for training and validating algorithms. Nevertheless, some notable datasets have enabled the development of advanced methods, such as NEU, Severstal: Steel Defect Detection, and DAGM. Among them, NEU stands out particularly in the literature as it contains images of six different types of defects, which are common across a wide range of industrial parts, making it a key reference for research in automatic surface defect detection.

The NEU dataset was first introduced in 2013 (Song & Yan, 2013). The images in the dataset are obtained after the cooling of steel sheets, when the surface temperature ranges between 500 and 700 °C. Originally, the images are in grayscale and have a resolution of 1024 × 1024 pixels; however, to optimize processing time, they are reduced to 200 × 200 pixels. In this work, a detection method based on a noise-robust feature descriptor, called AECLBP, is introduced, designed to improve the recognition rate in steel surface inspection. AECLBP is an enhancement of the conventional CLBP operator, which generates local binary codes to describe intensity differences in an image. Unlike CLBP, AECLBP uses an adjacent evaluation window that reduces sensitivity to noise, making it more resistant to interference. Experiments show that AECLBP offers greater robustness against additive noise compared to CLBP, making it a solid solution for identifying surface defects, even under adverse conditions.

Over the past decade, the NEU dataset has been used to train deep learning models for surface defect detection. For instance, in He et al. (2020), an innovative approach is proposed to create a surface defect detection network based on a convolutional architecture combined with a multiscale feature fusion network (MFN). This fusion enables the capture of information at different hierarchical levels of the network, improving the spatial representation of the defects and improving its identification. The main architecture of this system, called Deep Defect Network (DDN), utilizes pre-trained networks like ResNet34 and ResNet50, which offer significant advantages in terms of accuracy and efficiency. Additionally, the model implements a region proposal network (RPN), which generates regions of interest (ROI) where detection is focused. For each ROI, a classifier is applied to determine the class of the defect present. When evaluating the results using DDN with ResNet50, an AP_{50} of 0.823 was obtained, representing a 4.4% improvement compared to the results achieved with Faster R-CNN also using ResNet50.

The adaptation of pre-established networks, which have proven effective in general problems, is a common practice in improving specialized models. An example of this can be found in Kou et al. (2021), where YOLOv3 is modified to optimize surface defect detection using the NEU dataset. Among the model's main innovations is the incorporation of a feature selection mechanism without the use of anchor boxes, allowing for better feature learning and a reduction in inference time. Additionally, dense convolutional blocks were integrated, specifically designed to extract richer information, effectively improving the reuse and propagation of features as well as the network's representation capacity. To contrast the obtained results, various generalist models are used and compared with the proposed model. Notably, YOLOv3 achieves an AP_{50} of 0.453, while its modified version reaches an AP_{50} of 0.722. However, it is important to note that single shot detector (SSD) outperforms the proposed model, achieving an AP_{50} of 0.724.

Another notable example of adapting generalist networks for defect detection is the model forceful steel defect detector (FDD) (Akhyar et al., 2023), specifically designed to identify defects on steel surfaces. This model uses the Cascade R-CNN architecture as its foundation, enhancing it with deformable convolutions and deformable RoI pooling, allowing it to more accurately adapt to the irregular geometric shapes of the defects. Additionally, a guided anchoring mechanism is incorporated to generate more precise bounding boxes. The model also employs random scaling and final scaling techniques in the training and inference phases to improve image representation. The results, evaluated on the Severstal, NEU, and DAGM steel datasets, demonstrate that the proposed model achieves significant improvements compared to other approaches. The proposed model

achieves an AP_{50} of 0.834, surpassing the results obtained with the deep defect network (DDN) model by 1.1%.

In some scenarios, understanding the shape of defects, not just their presence, is essential. In Song et al. (2020), it is addressed this by adapting the NEU dataset for binary semantic segmentation, emphasizing defect contours through a saliency-based approach. Similarly, in Lv et al. (2025) it is proposed a Union Channel Pruning (UCP) algorithm to optimize U2Net for real-time segmentation on aluminum strips. By combining convolution weight magnitudes with activation expectations, the method effectively prunes 62.5% of channels, achieving superior accuracy and speed compared to other lightweight models on a custom industrial dataset.

The difference between object localization and understanding their shape is subtle. For this reason, in Usamentiaga et al. (2021), object detection and semantic segmentation methods are compared, using YOLOv5 and UNet algorithms as reference. This research addresses the challenge of adapting the dataset, which is a time-consuming task but essential for training semantic segmentation algorithms.

YOLOv5 has been widely used for surface defect detection. In Wang et al. (2023), YOLOv5 is enhanced through a multiscale feature fusion module (ATPF), which allows adaptive integration and weighting of features at different scales, improving semantic information extraction. This innovative improvement, combined with the use of CARAFE, BiFPN, and the modification of YOLOv5's C3 module with the CG2 module, where normal convolutions are replaced with Ghost Convolutions, results in improvements not only in terms of accuracy but also in inference speed. The proposed model, named CG-NET, achieves an AP_{50} of 0.759. When compared to other models, such as YOLOv5s, which reaches an AP_{50} of 0.696, it shows a 9% improvement. However, in Usamentiaga et al. (2022), YOLOv5s is also used, achieving an AP_{50} of 0.78.

Although YOLOv5 has been widely used, more recent versions are now employed in several works. For example, in Liu et al. (2025), YOLOv8 is used along with two global attention modules (GAM) to extract global feature information and establish long-range dependencies between defects. The first module, called GAM-A, is effective at extracting surface-level feature information, while the second, GAM-M, focuses on capturing deep and high-dimensional features. This work also employs a cascade feature fusion network (CFN) to flexibly merge features, allowing the model to adapt to different defect scales. Additionally, soft-NMS is used instead of NMS to avoid aggressive elimination of predictions, thus improving accuracy. This model, named GC-NET, achieves an AP_{50} of 0.771, representing a 5.4% improvement compared to the results obtained with YOLOv8 (unspecified version). However, as discussed in "Results and discussion" section, YOLOv8n can achieve an AP_{50} of 0.756, meaning that the improvement with GC-NET would only be 1.9%.

SOP-Net (Li, 2024) addresses the challenge of detecting small object defects in industrial settings by integrating two modules: the small object area perception (SOAP) and small object feature perception (SOFP). SOAP uses spatial and self-attention to focus on defect regions and reduce background interference, while SOFP enhances inter-class feature differentiation via dynamic weighting. Experiments on NEU dataset show consistent improvements over the YOLOv8n, with up to 2.9% gain in AP_{50}.

In Wang et al. (2024), a modified version of YOLO, named YOLO-AFK, is proposed. This approach introduces a Fusion Attention Network (FANet) to enhance the detection of small objects, and Alternable Kernel Convolution (AKConv), which adapts convolutional shape and size to defect characteristics. Experimental results report an AP_{50} of 0.79, outperforming YOLOv5 and YOLOv8, which achieve 0.73 and 0.74 respectively, indicating a significant performance improvement.

RDD-YOLO (Zhao et al., 2023), based on YOLOv5, is proposed for steel surface defect detection. The model features a Res2Net backbone for better feature extraction at various scales, a double Feature Pyramid Network (DFPN) in the neck for enhanced representation generation, and a decoupled head for separating regression and classification tasks to improve detection precision. Experimental results demonstrate that RDD-YOLO outperforms YOLOv5, achieving 0.811 AP_{50} on the NEU-DET dataset.

Table 1 summarizes the results obtained from various proposals for surface defect detection using the NEU dataset. This analysis evaluates nine methods proposed over the last five years. Interestingly, the Deep Defect Network (DDN) model from 2020 achieves a higher mAP than more recent models, such as the modified YOLOv8 (GC-NET) from 2024 or YOLO-AFK and RDD-YOLO from 2025. Only the forceful steel defect detector (FDD) from 2023 shows slightly better performance. It is important to note that the same data partitions were not used across the evaluated studies, and that inference time measurements were conducted on different hardware setups specific to each work.

These results raise several questions: Why do advancements not reflect a consistent improvement year after year? How is it possible for two studies using the same model to obtain such disparate results? As discussed in "Results and discussion" section, the way the dataset is split significantly impacts the results, especially when working with small datasets. To rigorously compare the results of different studies, it is essential that all algorithms be trained and evaluated using the same images. Otherwise, a fair comparison is

Table 1 Summary of surface defect detection performance across various studies using the NEU dataset

Model	Year	AP_{50}	AP_{50} improvement	Test split	Training split	Inference time (ms)
Faster RCNN—VGG16 (He et al., 2020)	2020	0.723	–	30%	70%	–
Faster RCNN—ResNet50 (He et al., 2020)	2020	0.748	–	30%	70%	–
DDN (He et al., 2020)	2020	0.823	0.075	30%	70%	–
YOLOv3 (Kou et al., 2021)	2021	0.453	–	11%	89%	16.1
Modified YOLOv3 (Kou et al., 2021)	2021	0.722	0.269	11%	89%	15.6
Faster RCNN (Usamentiaga et al., 2022)	2021	0.710	–	25%	75%	37.0
SSD (Usamentiaga et al., 2022)	2021	0.720	–	25%	75%	29.0
YOLOv5 (Usamentiaga et al., 2022)	2021	0.780	0.060	25%	75%	6.8
CP-YOLOv3-dense (Akhyar et al., 2023)	2023	0.767	–	30%	70%	–
DDN (Akhyar et al., 2023)	2023	0.823	–	30%	70%	–
FDD (Akhyar et al., 2023)	2023	0.834	0.011	30%	70%	83.3
YOLOv3 (Wang et al., 2023)	2023	0.731	–	10%	90%	25.0
YOLOv5s (Wang et al., 2023)	2023	0.696	–	10%	90%	10.4
GhostNet-YOLOv5 (Wang et al., 2023)	2023	0.732	–	10%	90%	13.5
CG-Net (Wang et al., 2023)	2023	0.759	0.027	10%	90%	9.5
YOLOv8 (Liu et al., 2025)	2024	0.733	–	–	–	19.6
YOLOv9 (Liu et al., 2025)	2024	0.735	–	–	–	21.3
GC-Net (Liu et al., 2025)	2024	0.771	0.036	–	–	25.0
YOLOv8n (Li, 2024)	2024	0.756	–	–	–	–
SOP-Net (Li, 2024)	2024	0.779	0.023	–	–	–
YOLOv5 (Wang et al., 2024)	2025	0.734	–	20%	80%	–
YOLOv8 (Wang et al., 2024)	2025	0.743	–	20%	80%	–
YOLO-AFK (Wang et al., 2024)	2025	0.792	0.049	20%	80%	24.4
YOLOv3 (Zhao et al., 2023)	2025	0.723	–	20%	80%	33.3
YOLOv5l (Zhao et al., 2023)	2025	0.768	–	20%	80%	9.9
YOLOvX (Zhao et al., 2023)	2025	0.793	–	20%	80%	9.9
RDD-YOLO (Zhao et al., 2023)	2025	0.811	0.018	20%	80%	17.3

not possible. Furthermore, there is a random variable in the training process that can cause variations in the results. This variability is especially noticeable in small and unbalanced datasets, such as those typically used in surface defect detection. Therefore, to accurately replicate the experiments, it is essential that the code for the proposed architectures be published, along with the evaluation methodology used.

Methodological proposal for model comparison: cross-validation and statistical analysis

The comparison of surface defect detection methods in images is a recurring challenge due to the inherent variability in datasets and experimental setups. Although many works in the literature use the same dataset, they do not always specify the exact partitions between the training and testing sets. This can significantly influence the reported results, specially in small datasets, making it difficult to draw direct comparisons between different proposals. Even small improvements in metrics such as average precision (AP), on the order of 1–2%, may not be significant if these sources of variability are not controlled.

- Problem of fixed partitions: random or unreported partitions introduce evaluation bias, as models may benefit from or be hindered by particular characteristics of the data included in the testing set. This is especially critical in datasets with a limited number of images or when the classes are imbalanced. In such cases, a single partition may not be representative of the global data distribution.
- Relevance of differences: small variations in metrics, such as average precision (AP), may result from random factors rather than genuine model improvements. Without proper statistical analysis, it becomes challenging to determine whether these differences are attributable to the evaluated methods, the randomness introduced by dataset partitions, or simply noise. The learning process inherently involves stochastic elements, which can lead to varying results.

To overcome these limitations, the methodology shown in Fig. 1 is proposed. Its objective is to enhance the robustness of scientific studies and promote the reproducibility of results. The methodology can be applied through the following steps:

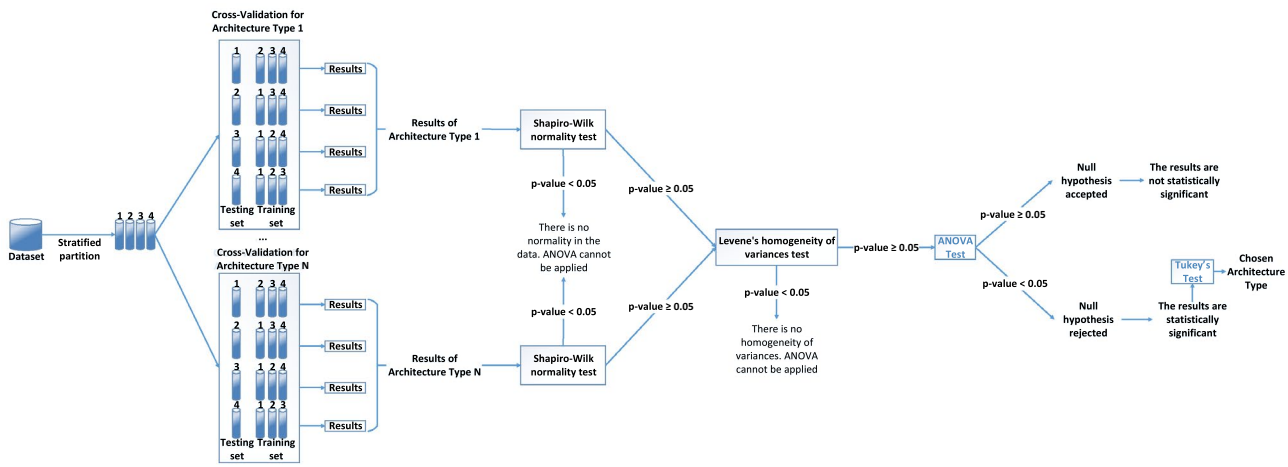


Fig. 1 Proposed methodology for model comparison. The methodology consists of six phases. (1) A stratified split of the dataset is performed to ensure the independence of observations. (2) The different architectures are trained and evaluated. (3) The Shapiro–Wilk normality test is applied to the results of each architecture. If the p value is less than 0.05, the data do not follow a normal distribution, and therefore ANOVA cannot be applied. (4) Levene's test is used to assess the homogeneity of variances. If the p value is less than 0.05, the variances are not homogeneous, and thus ANOVA is not valid. (5) ANOVA is conducted to determine whether there are statistically significant differences between the models. If the p value is less than 0.05, it is concluded that there are significant differences between at least two groups. (6) Tukey's multiple comparisons test is applied to identify which models exhibit significant differences and to select the architecture with the most statistically robust performance

1. Stratified partitioning: in all datasets, there are objects whose detection is easier than others, but in small datasets, such objects can drastically influence the results. If a large proportion of easy-to-detect objects is used in the test set, the evaluation results might not be realistic, as a production system will encounter both easy and difficult objects to detect (Chen et al., 2025; Xie et al., 2025; Yang et al., 2024). For this reason, it is proposed to divide the dataset into four equally sized stratified partitions, ensuring that each partition contains a representative proportion of each defect type. Stratified partitioning is a sampling technique that preserves the class distribution across all subsets. This helps prevent bias due to over- or under-representation of certain classes in any given partition, which is especially critical in small and imbalanced datasets.
2. Cross-validation: the results can vary depending on which partitions are used for training and testing, which is especially relevant in small datasets. To address this variability, it is proposed to use four-fold cross-validation. This ensures that each partition of the dataset is used both for training and testing the model, providing a more robust and representative estimate of the model's performance. The choice of using four folds is motivated by the need to balance evaluation reliability with training data sufficiency. Using too few folds may reduce the statistical validity of the comparison, while too many folds would shrink the training subsets and increase computational cost without a proportional improvement in robustness. Empirical observations confirmed that four folds offer an optimal trade-off between model stability, computational efficiency, and statistical reliability (especially important in scenarios with limited and imbalanced data).
3. Shapiro–Wilk test: One of the assumptions required by ANOVA is the normality of the data. To verify this assumption, the Shapiro–Wilk test is applied, which formulates the following hypotheses:

   - Null hypothesis (H_0): the data follow a normal distribution.
   - Alternative hypothesis (H_1): the data do not follow a normal distribution.

   If the resulting p value is less than 0.05, the null hypothesis is rejected, indicating that the data are not normally distributed and ANOVA should not be applied. Otherwise, the null hypothesis is accepted and the analysis can proceed with ANOVA.
4. Levene's test: Another assumption required by ANOVA is the homogeneity of variances across all groups. To verify this, Levene's test is applied, which formulates the following hypotheses:

   - Null hypothesis (H_0): the variances are equal (homogeneity of variances).
   - Alternative hypothesis (H_1): the variances are not equal.

If the resulting p value is less than 0.05, the null hypothesis is rejected, indicating that the assumption of homogeneity of variances is violated and ANOVA should not be applied. Otherwise, the null hypothesis is accepted and the analysis can proceed with ANOVA.

5. Analysis of variance (ANOVA) test: Once the necessary assumptions have been verified, the ANOVA test can be applied to determine whether there are statistically significant differences between the evaluated models or architectures. To ensure the independence of observations, the results (AP_{50}) obtained from the four folds of the cross-validation process are used. This is particularly important in small datasets, as opposed to large-scale datasets such as MS COCO (Lin et al., 2014), where a greater amount of data helps reduce the impact of intrinsic variability. Even when using the same training data, two models may yield different results when evaluated on the same set due to stochastic elements in the training process. This variability introduces uncertainty into the results, meaning that a small improvement in AP_{50} may not be statistically significant.
   To address this, an ANOVA test is proposed to determine whether the differences observed between models are truly significant or if they are the result of random variability. The ANOVA test is a statistical procedure used to compare the means of three or more groups and determine if at least one is significantly different from the others. It compares the variability within groups with the variability between groups, aiming to see if the observed differences in means are large enough to not be explained solely by random variability. Specifically, a one-way ANOVA test is used to compare model performance (AP_{50}) across multiple groups defined by a single categorical factor, with all other experimental conditions held constant. This type of ANOVA is appropriate when only one independent variable is considered.
   The ANOVA test formulates the following hypotheses:

   - Null hypothesis (H_0): there are no statistically significant differences between the models.
   - Alternative hypothesis (H_1): there are statistically significant differences between the models.

   If the resulting p value is less than 0.05, the null hypothesis is rejected, indicating that there are statistically significant differences between at least two of the evaluated models or architectures. Otherwise, the null hypothesis is accepted, meaning that no model shows statistically significant improvements over the others.
6. Tukey's test: in the case that the ANOVA's null hypothesis is rejected and the alternative hypothesis is accepted, it is necessary to determine which model(s) showed statistically significant differences. For this, a Tukey test is applied, a post-hoc procedure conducted after the ANOVA analysis. The purpose of the Tukey test is to identify which specific groups have significant differences between them after rejecting the null hypothesis of equal means.
   Tukey's test formulates the following hypotheses for each pairwise comparison:

   - Null hypothesis (H_0): the means of the two models/architectures are equal (no statistically significant difference).
   - Alternative hypothesis (H_1): the means of the two models/architectures are different (there is a statistically significant difference).

   Tukey's test compares each model to every other model, performing pairwise comparisons between all groups in the dataset. This means that for every model/architecture, it tests the null hypothesis by comparing its performance against the others in the dataset.
   When ANOVA indicates that at least one group differs, the Tukey test evaluates all possible pairwise comparisons, while controlling for family-wise error rate (FWER). This correction helps to avoid Type I errors (false positives) when multiple comparisons are performed. If the p value from Tukey's test for a particular comparison is less than 0.05, the null hypothesis is rejected, indicating that there is a statistically significant difference between the two models. In this case, and after analyzing which model/architecture has a higher AP_{50} it will be possible to state that one model is better than another.

Materials and methods

Dataset

The dataset used in this study is the Northeastern University (NEU) surface defect dataset, as it is one of the most widely used benchmarks in the literature on surface defect detection. It is publicly available, relatively small, balanced and well-suited for rapid experimentation. Moreover, the NEU dataset offers different variants tailored for image classification, object detection, and segmentation tasks. It contains 1800 grayscale images captured after the cooling process of steel sheets, and it categorizes six types of surface defects.

- Crazing: corresponds to a network of visible cracks on a metallic surface, caused by tensile stress. These cracks extend in specific areas, leading to small fractures.
- Inclusion: defects formed by compounds such as oxides and sulfides, created during the melting process. Inclusions are common in all metallic alloys.
- Patch: imperfections caused by a surface oxidation process.
- Pitted surface: a localized form of corrosion that randomly creates small holes in the metal.
- Rolled-in scale: occurs when mill scale, a mixture of iron oxides, gets embedded in the metal during the rolling process.
- Scratch: straight lines parallel to the rolling direction of the steel, caused by foreign objects.

Figure 2 shows a representative example of each of these defects.

Object detection algorithms

Object detectors based on deep learning can be classified into two main types:

- Single-stage detectors: These detectors simultaneously perform object identification and localization within an image. Their primary advantage lies in their speed, as they combine both processes into a single stage. However, their accuracy may be limited in certain domains, particularly when detecting small objects or in complex scenes. The most well-known examples of this type of detector are YOLO (You Only Look Once) and SSD (Single Shot Detector).
- Two-stage detectors: This approach first generates region proposals of interest (ROI) and then, in a second stage, classifies and refines the detections within those regions. Their greatest advantage is their accuracy, specially for detecting small objects or in more complex situations, although this comes at a higher computational cost. The most representative model of this category is Faster-RCNN.

In recent years, there has been a trend toward the use of single-stage detectors, driven by the evolution of YOLO models. The basic functioning of YOLO involves dividing the image into a grid, where each cell is responsible for detecting objects whose centers fall within it. After making predictions, non-maximum suppression (NMS) is applied to eliminate redundancies, as multiple detections are generated (Redmon et al., 2016).

Since the first version of YOLO, various improvements have been implemented. In YOLOv2 (Redmon & Farhadi, 2017), anchor boxes were introduced to leverage the fact that many objects in datasets have similar shapes and sizes. These serve as initial estimates for object detection, which are then refined.

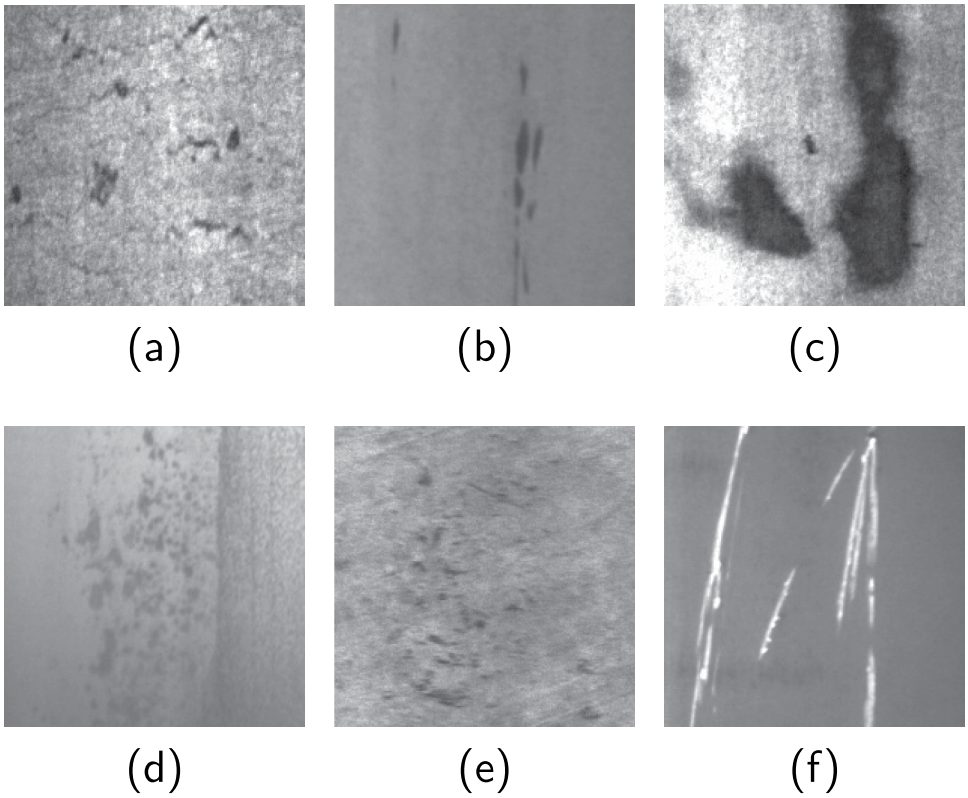


Fig. 2 Types of defects in the NEU dataset: **a** crazing, **b** inclusion, **c** patch, **d** pitted surface, **e** rolled-in scale and **f** scratch

YOLOv3 (Redmon & Farhadi, 2018) introduced predictions at three distinct scales to enhance the detection of small objects, resulting in a notable increase in the accuracy of identifying these objects. However, this improvement also led to an increase in inference time, which can affect the speed of the detection process in real-time applications. This trade-off between accuracy and efficiency is a constant challenge in the development of object detection models.

Subsequent versions, YOLOv4 (Bochkovskiy et al., 2020) and YOLOv5 (Jocher, 2020), incorporated significant advancements. The backbone adopted the CSPNet (Cross-Stage Partial Network) architecture, resulting in a faster and more accurate model. Additionally, PANet (path aggregation network) was integrated to facilitate the transfer of the most relevant features from the backbone to the head, which continues to utilize the YOLOv3 architecture. Among the data augmentation techniques applied, Mosaic Augmentation stands out, as it generates new images from fragments of several images, enriching the training process.

In YOLOv6 (Li et al., 2022), anchor boxes were eliminated, and instead, strategic key points in the image are detected to identify objects. This allows for greater versatility in detecting objects of varying sizes and shapes, although training becomes more complex. Furthermore, YOLOv6 enhances efficiency with an EfficientRep design and utilizes a Rep-PAN structure to effectively integrate features, achieving a balance between speed and accuracy.

On the other hand, YOLOv7 (Wang et al., 2023) introduces an extended efficient layer aggregation network in its backbone, improving both inference speed and accuracy. It implements an "expand, mix, and merge" strategy to enhance the model's learning capacity without affecting the gradients required for training.

YOLOv8 (Chien et al., 2024), in turn, stands out for providing an optimized user experience and integrating all previous advancements. With five pre-trained models on MS COCO, it is particularly well-suited for industrial applications and achieves outstanding performance with minimal post-training adjustment. Figure 3 shows YOLOv8 architecture for its five models.

The series continues to evolve with YOLOv9 (Wang et al., 2024b), which focuses on enhancing accuracy and efficiency through innovations in network structure and information processing. Inspired by concepts like Programmable Gradient Information (PGI), YOLOv9 aims to prevent data loss during multiple layers of feature extraction, optimizing information flow. This new version incorporates the GELAN (generalized efficient layer aggregation network) architecture to enhance gradient planning and parameter utilization, allowing even lightweight models to deliver superior performance.

YOLOv10 (Wang et al., 2024a) is designed to improve the balance between performance and efficiency in real-time object detection. Unlike its predecessors, it introduces a novel approach that eliminates the reliance on non-maximum suppression (NMS), enabling faster and simpler inference for end-to-end implementations. Additionally, it optimizes various model components through a comprehensive strategy that enhances both efficiency and accuracy, reducing computational redundancy and the number of parameters.

Lastly, YOLOv11 (Jocher & Qiu, 2024) represents the most recent version in the YOLO series. This update consolidates the latest advancements in object detection and integrates them into a platform designed to optimize the user experience. Its innovative design makes it ideal for facilitating multiple research endeavors, allowing researchers to explore new applications and improvements in the field of computer vision.

Evaluation metrics

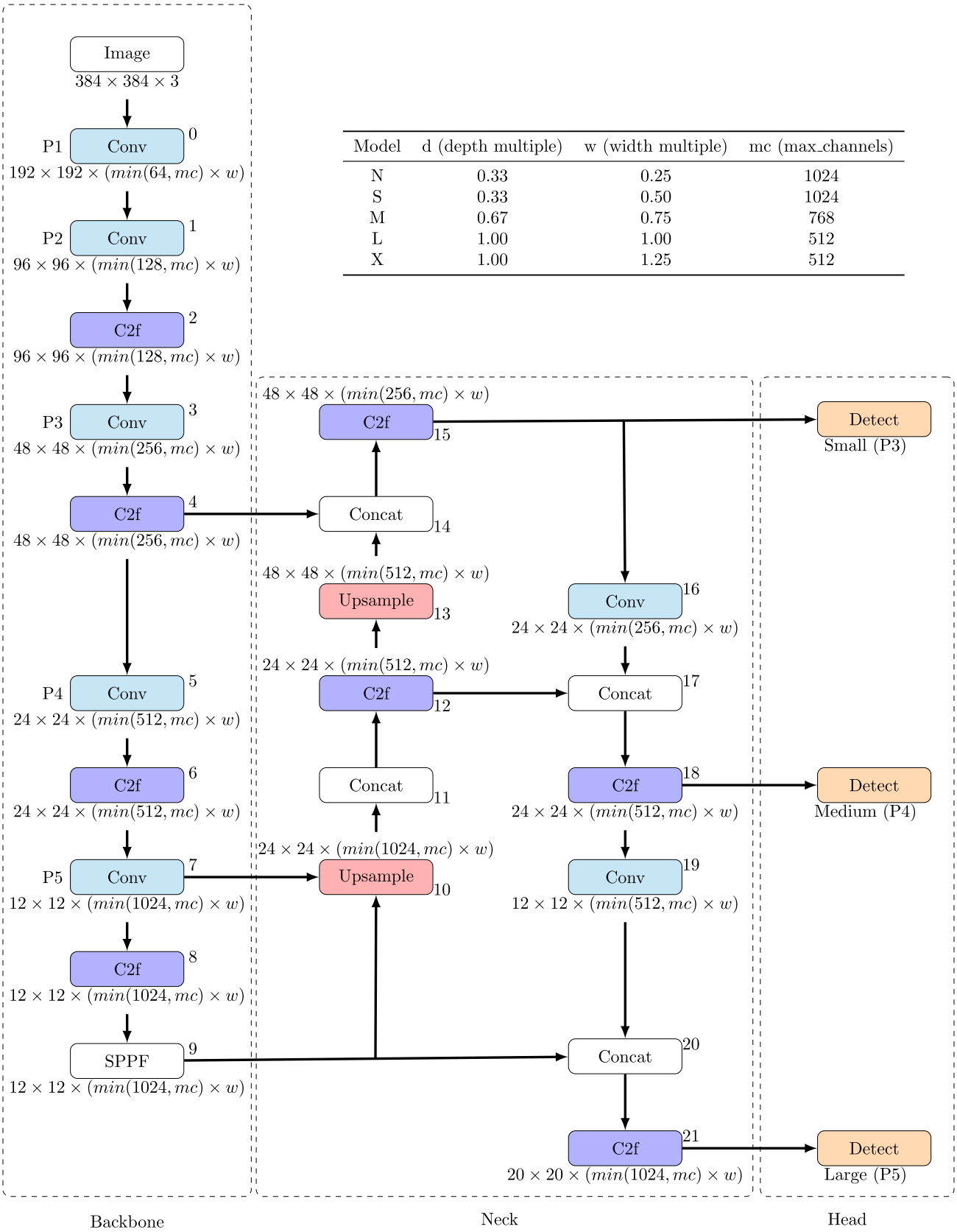
To compare the results of different object detection models, it is essential to clearly define the metrics used. The fundamental metrics upon which final evaluations are built include:

- True positives (TP): Correctly identified objects.
- False positives (FP): Incorrectly identified objects.
- False negatives (FN): Objects that were not detected when they should have been.
- True negatives (TN): Correctly non-detected objects. However, in object detection, TNs are not used because theoretically, there would be an infinite number of them in each image.

To apply these basic metrics, it is necessary to define what constitutes a correct prediction. In object detection, a prediction is considered correct if its overlap with the ground truth exceeds a certain Intersection over Union (IoU) threshold, defined in Eq. 1.

Based on these basic metrics, more advanced ones can be derived:

- Precision: Measures the percentage of correct predictions out of the total predictions made (defined in Eq. 2).
- Recall: Assesses the percentage of correctly detected objects out of the total objects that should have been detected (defined in Eq. 3).
- F1 Score: Combines precision and recall into a single metric, calculating the harmonic mean between the two (defined in Eq. 4).



Model	d (depth multiple)	w (width multiple)	mc (max_channels)
N	0.33	0.25	1024
S	0.33	0.50	1024
M	0.67	0.75	768
L	1.00	1.00	512
X	1.00	1.25	512

Fig. 3 YOLOv8 architecture differentiating its three parts: backbone, neck and head [Figure adapted from Ultralytics (2023)]

While these metrics are useful in image classification or segmentation, object detection must also consider prediction confidence. Some models may perform better at certain confidence levels than others. To account for this variability, average precision (AP) is used, calculated from the precision–recall curve across all confidence levels. AP is the area under this curve, allowing for the comparison of models without depending on a specific confidence threshold.

Finally, AP can be computed using different IoU thresholds. For example, when using an IoU threshold of 0.5, the metric is referred to as AP_{50}.

For the purpose of evaluating object detection performance in this study, the metric chosen is AP_{50}, which provides a reliable measure of model effectiveness in detecting objects with a minimum overlap of 50%. This threshold strikes a balance between precision and recall, offering a clear evaluation metric for comparing different models while accounting for detection accuracy at a reasonable level of IoU. AP_{50} has become a standard in object detection benchmarks, making it an appropriate choice to ensure consistency and comparability across different models and datasets.

$$IoU = \frac{|D \cap G|}{|D \cup G|} \tag{1}$$

$$Precision = \frac{TPs}{TPs + FPs} \tag{2}$$

$$Recall = \frac{TPs}{TPs + FNs} \tag{3}$$

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{4}$$

Defect detection models to be evaluated

In "Literature review" section, various studies proposing different methods for detecting defects in rolled steel strips have been analyzed. However, due to the large volume of existing research, it is not feasible to evaluate all of them in detail. Therefore, three representative approaches related to defect detection have been selected and interpreted for detailed evaluation. In "Results and discussion" section, a thorough comparison of the results of these models is conducted.

- DefectDef (Medak et al., 2022): a deep learning-based object detector is carried out to detect defects in ultrasonic images. This approach utilizes the typically three-part architecture: backbone, neck and head. The backbone follows an encoder–decoder scheme, similar to UNet but with key modifications:

  - Multiple blocks are not used for each resolution level.
  - The number of filters does not increase as the resolution decreases.
  - Batch normalization and dropout are applied for regularization.

  Additionally, the Swish activation function is implemented to enhance gradient flow. The method downsamples the image using convolutions and max-pooling. Subsequently, the feature maps are upsampled to be concatenated with the encoder feature maps at matching resolutions. After recovering the original resolution, a second downsampling is performed to generate the feature maps, which, after being combined through a biFPN neck (P3–P7), serve as input for the detection head.

  Figure 4 illustrates the implementation developed based on the study's interpretation. The backbone is composed of two main parts. The first part follows an encoder–decoder structure similar to UNet (Ronneberger et al., 2015). In each encoding block, the image's dimensionality is reduced by half, while consistently maintaining 32 filters. This block includes a 3×3 convolution with a stride of 1 and padding of 1, followed by batch normalization, a SiLU (also known as Swish) activation function, a dropout layer, and max pooling.

  Each decoding block is preceded by an upsample and concatenation with the feature map from the corresponding encoding block. The decoding stage uses a SiLU activation, a 1×1 convolution with a stride of 1 and no padding, followed by batch normalization and another SiLU activation.

  The second part of the backbone begins with half the resolution of the original image. Unlike in models like YOLOv8, the number of filters remains constant at 64. From there, a series of convolutions (with batch normalization and SiLU activation) is applied to build the feature pyramid network (FPN). The network's neck follows a biFPN structure to refine the features passed to the detection heads. Finally, the YOLOv8 detection heads are used to generate predictions at five scales. After applying Non-Maximum Suppression (NMS), the results are combined.

- YOLOv8-Unet: after analyzing the previous implementation, the idea of combining the capabilities of UNet and YOLOv8 emerged to enhance defect detection. Figure 5 illustrates the outcome of this approach. The network consists of two main components:

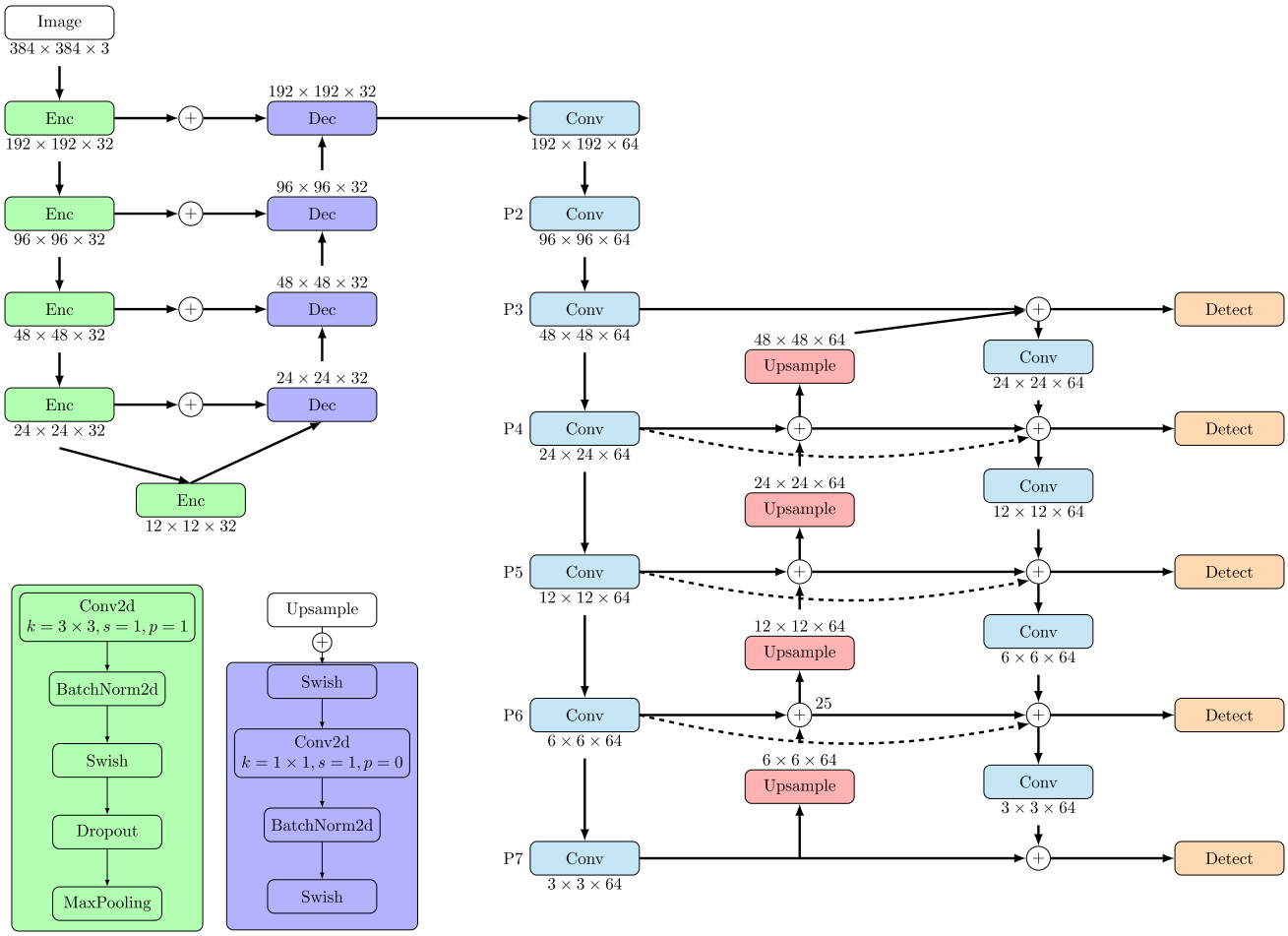


Fig. 4 Interpretation of the DefectDef architecture proposed in Medak et al. (2022) combined with YOLOv8

- UNet: in this part, as in the original implementation, two convolutions followed by max pooling are used to reduce the dimensionality of the input image. After each convolution, a batch normalization layer is applied, along with the SiLU activation function. Unlike the previous implementation and in line with the original concept, the number of filters increases progressively in each encoding block, starting with 64 and reaching up to 1024 filters. The decoding stage remains similar, with the goal of restoring the resolution of the feature maps while gradually reducing the number of filters to select the most relevant features.
- YOLOv8n: once the UNet backbone is completed, the results are merged with the YOLOv8n backbone. The neck follows the PANet structure, as in YOLOv8, which facilitates the aggregation of essential features to improve precision. For predictions, the YOLOv8 detection heads are employed, performing detections across three different scales, thus optimizing object detection capabilities for varying object sizes.

This combination of architectures aims to maximize precision and efficiency in defect detection by leveraging the strengths of both networks.

- Background suppression module (Song et al., 2023): the last option to consider is the implementation of a background noise suppression module. Its goal is to improve the contrast between defects and the background, thus optimizing the accuracy of surface defect detection. Since the proportion of defects in an image is small, the global average value tends to be closer to that of the defect-free areas. Therefore, by adding corrections both to the difference between the defect-free areas and the average, as well as to the difference between the defective areas and the average, the distance between them increases, making defect detection easier. In the original work, Faster R-CNN combined with ResNet50 as a backbone was used, to which a noise suppression module was added. Additionally, deformable convolutions

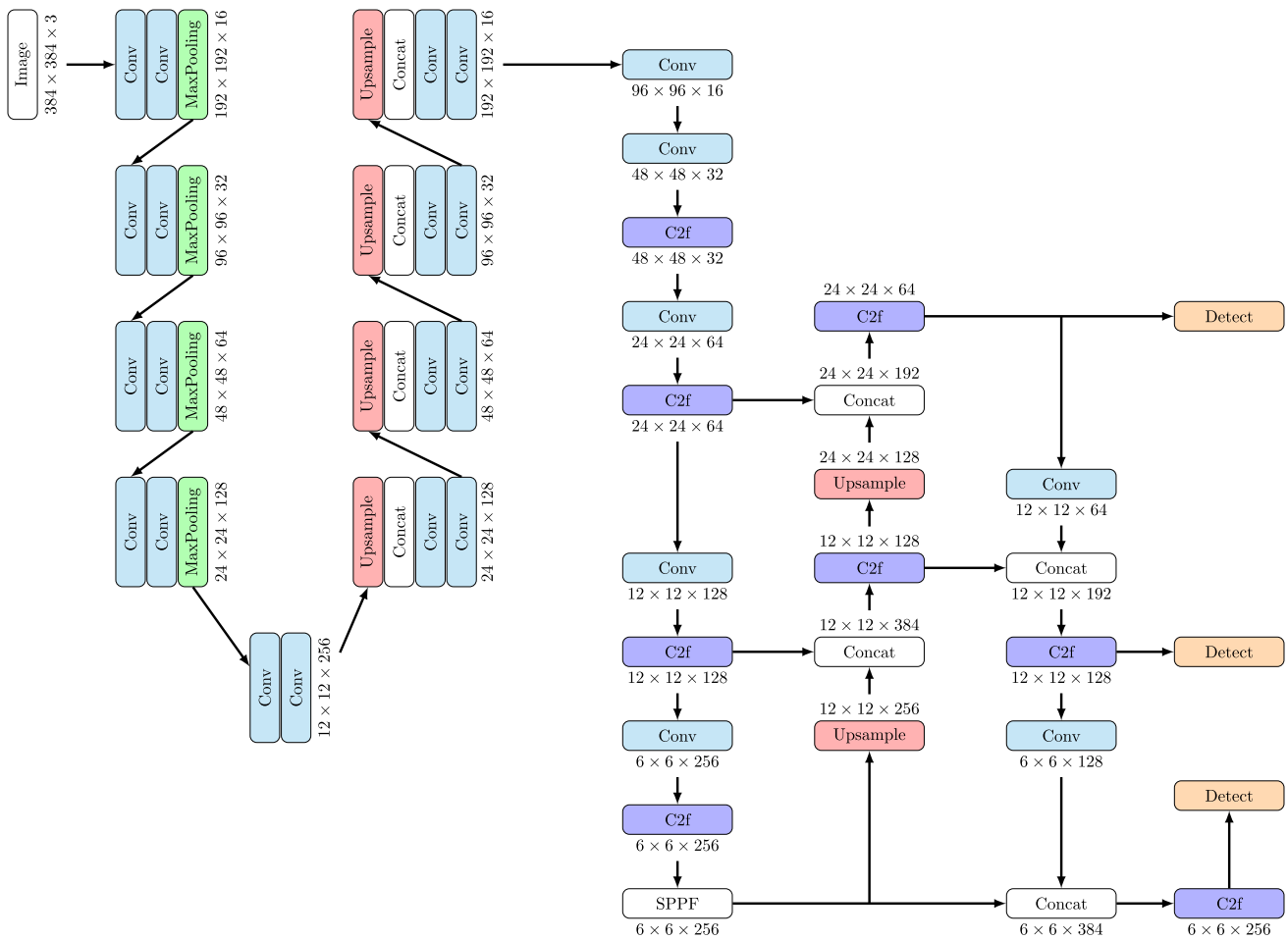


Fig. 5 Architecture based on the combination of YOLOv8 and UNet [Figure adapted from Ultralytics (2023) and Ronneberger et al. (2015)]

were incorporated (Dai et al., 2017; Zhu et al., 2019) due to the irregular shape of the defects. In the implementation proposed in this work, it was decided to combine deformable convolutions and the background noise suppression module with YOLOv8, as it has shown better results than Faster R-CNN. Figure 6 shows the proposed architecture.

Experimental setup

The Northeastern University (NEU) surface defect dataset was used for model evaluation, as it is widely recognized in the literature for surface defect detection tasks. The dataset contains 1800 grayscale images of steel sheets, categorized into six types of defects: cracks, scratches, pits, patches, stains, and inclusions. These images were captured after the cooling process of the sheets, representing common industrial defects.

Experiments were conducted on a high-performance server equipped with an 11th Gen Intel(R) Core(TM) i9-11900K processor at 3.50 GHz and an NVIDIA GeForce RTX 3090 GPU. This setup ensures the necessary computational power to manage the demands of deep learning model training and evaluation.

The following models were evaluated:

- Five versions of YOLOv8, with different configurations and hyperparameters
- Five versions of YOLOv10, including the latest iterations with optimizations
- Five versions of YOLOv11, focusing on various enhancements to the model architecture
- Three custom implementations, inspired by recent works in the literature, designed for surface defect detection

In all experiments, 75% of the dataset was used for training, and 25% for testing, ensuring that the class distribution was preserved in both splits. This data partitioning ensures a balanced approach to model evaluation. To validate the reliability of the results, the proposed four-fold stratified cross-validation method was applied. This methodology

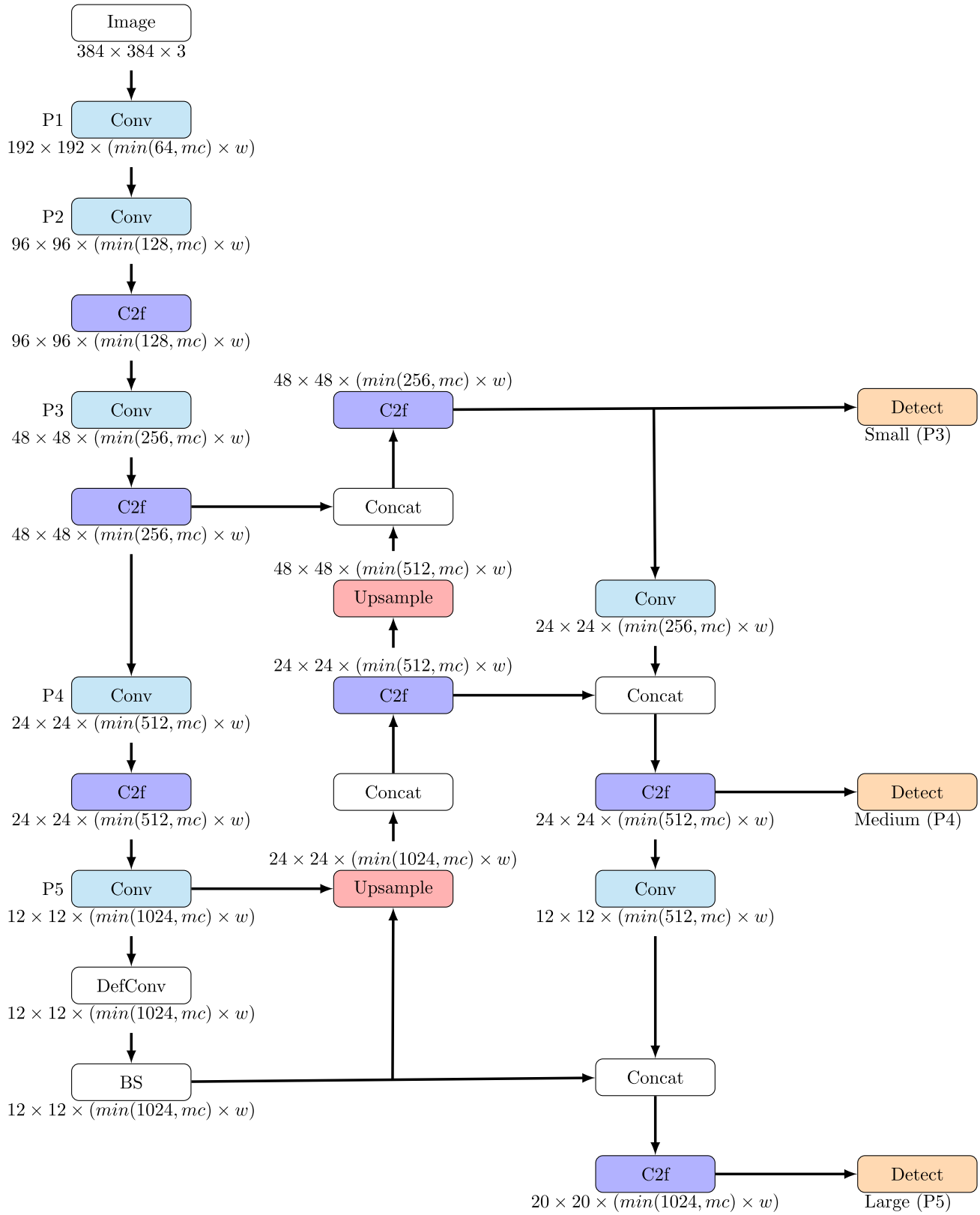


Fig. 6 YOLOv8 with background suppression (BS) module and deformable convolutions [Figure adapted from Song et al. (2023)]

Table 2 YOLOv8n results on NEU varying the number of images used for testing and training

Percentage of images for testing (%)	Percentage of images for training (%)	AP_{50}
20	80	0.780
25	75	0.766
30	70	0.736

ensures that each fold maintains a representative distribution of defect types, thus minimizing the bias introduced by easier-to-detect defects.

The stratified partitioning process followed in this study can be described in four main steps:

- Class-wise grouping: Initially, all instances are grouped according to their respective classes (i.e., defect types). This step ensures that instances belonging to the same class are processed collectively, enabling accurate tracking of class distributions.
- Division into folds: The dataset is then divided into K folds. Unlike random partitioning, the stratified approach maintains the original class distribution across all folds. Consequently, each fold reflects the overall distribution of defect types present in the complete dataset.
- Proportional allocation: For each fold, the number of instances assigned from each class is proportional to the class's frequency in the full dataset. For example, if a particular class accounts for 25% of the total instances, approximately 25% of the instances in each fold will belong to that class.
- Intra-class randomization: To avoid ordering bias, the instances within each class are randomly shuffled prior to fold assignment. This ensures that the final partitions are both representative and randomized, enhancing the robustness and reliability of the cross-validation process.

Results and discussion

The performance of models can vary significantly depending on the data used for training and testing, a particularly relevant issue in small datasets. This is common in tasks such as surface defect detection, where datasets are typically limited. A notable example is the NEU dataset, widely used in this field, which contains only 1800 images.

Table 2 presents the results obtained by applying YOLOv8n using different partitions of the NEU dataset. These partitions involve varying proportions of images for training and testing, allowing us to observe how the distribution of data affects model performance. The results show that performance, measured by AP_{50}, changes dramatically depending on the percentage of images used for testing. For instance, using 20% of the images for testing yields an AP_{50} of 0.780, which is 4.4% higher compared to using 30% for testing, where the AP_{50} drops to 0.736.

Table 3 Results of YOLOv8n on NEU varying the data partition used for testing and training

Testing set	Training set	AP_{50}
1	2, 3 and 4	0.766
2	1, 3 and 4	0.762
3	1, 2 and 4	0.753
4	1, 2 and 3	0.742

This highlights the importance of using consistent partitions when comparing results across different studies. Otherwise, apparent improvements or declines in model performance may be solely attributed to differences in data partitioning rather than intrinsic changes in the proposed models.

Another important issue that arises is the choice of data partitioning. Even when the same percentage of images is used for testing and training, the results can vary depending on the images selected for each task. Table 3 presents the results obtained by applying YOLOv8n under different partition configurations, where 25% of the images are used for testing and 75% for model training. For example, when performing a stratified division in which the dataset is split into four equal parts, with one subset used for testing and the remaining three for training, the results vary significantly. As observed, depending on the partition used, the results can differ by up to 2.4%.

For this reason, it is essential to consider the results from all partition configurations when selecting the model to ensure that the outcomes are statistically significant and not biased by the chosen partition.

The primary objective is to identify the model that best fits a specific dataset. In this study, the NEU dataset is used for detecting surface defects. To address this task, the proposed methodology is applied. The dataset is divided into four stratified partitions. In each iteration, one partition is used for evaluation, while the remaining three are used for training the model. This approach enables the evaluation of several models under controlled conditions. Table 4 presents the results for each of the architectures/models evaluated using four-fold cross-validation, along with the mean values. Based on these results, one might conclude that, for example, YOLOv5m ($AP_{50} = 0.765$) outperforms YOLOv10n ($AP_{50} = 0.733$) or YOLOv10s ($AP_{50} = 0.732$), or that YOLOv8n–BackgroundSuppression ($AP_{50} = 0.756$) performs better than YOLOv10n ($AP_{50} = 0.733$). But is this truly the case? Are the improvements observed statistically significant?

To determine the statistical significance of these differences, an one-way ANOVA test is performed. The analysis yields a p value of 0.0022 (less than 0.05), leading to

Table 4 AP_{50} for each architecture/model evaluated during the four-fold cross-validation, along with the mean value. Note: YOLOv8n–BS refers to YOLOv8n–BackgroundSuppression, and YOLOv8n–DD to YOLOv8n–DefectDef

Model	AP_{50}	AP_{50}	AP_{50}	AP_{50}	Mean AP_{50}
YOLOv5n	0.770	0.767	0.747	0.747	0.758
YOLOv5s	0.769	0.776	0.750	0.747	0.761
YOLOv5m	0.769	0.769	0.762	0.759	0.765
YOLOv5l	0.775	0.767	0.742	0.744	0.757
YOLOv5x	0.764	0.762	0.767	0.751	0.761
YOLOv8n	0.766	0.762	0.753	0.742	0.756
YOLOv8s	0.760	0.755	0.743	0.746	0.751
YOLOv8m	0.764	0.752	0.741	0.741	0.749
YOLOv8l	0.748	0.757	0.727	0.736	0.742
YOLOv8x	0.737	0.756	0.740	0.749	0.746
YOLOv8–Unet	0.773	0.748	0.744	0.718	0.746
YOLOv8n–BS	0.775	0.764	0.751	0.735	0.756
YOLOv8n–DD	0.313	0.326	0.375	0.451	0.366
YOLOv10n	0.746	0.744	0.715	0.725	0.733
YOLOv10s	0.739	0.745	0.719	0.723	0.732
YOLOv10m	0.744	0.747	0.732	0.747	0.743
YOLOv10l	0.748	0.747	0.731	0.736	0.741
YOLOv10x	0.742	0.733	0.731	0.732	0.735
YOLO11n	0.755	0.767	0.749	0.751	0.756
YOLO11s	0.763	0.755	0.745	0.732	0.749
YOLO11m	0.753	0.763	0.748	0.744	0.752
YOLO11l	0.762	0.763	0.752	0.757	0.759
YOLO11x	0.754	0.760	0.754	0.729	0.749

the rejection of the null hypothesis and confirming that the observed differences are statistically significant.

To identify which models present these significant differences, Tukey's test is applied. The results indicate that all models, except YOLOv8n–DefectDef, deliver significantly better results. This is because YOLOv8n–DefectDef performs substantially worse than the rest. Consequently, the detected significant differences are primarily due to the inclusion of this model, which serves as a clear underperforming benchmark.

When the methodology is repeated without including YOLOv8n–DefectDef, the results change significantly. In this case, most of the differences between the remaining models are not statistically significant, showing that even a 2.4% improvement in AP_{50} may lack statistical relevance. However, statistically significant differences were observed between specific architectures: YOLOv10n showed significant differences compared to YOLOv5m (p value $= 0.0386 < 0.05$), and YOLOv10s also showed significant differences compared to YOLOv5m (p value $= 0.0273 < 0.05$). These highlights that while many observed improvements may not be meaningful, certain architectural changes result in statistically supported performance differences, allowing to conclude that YOLOv5m outperforms YOLOv10n and YOLOv10s on the NEU dataset. However, it cannot be concluded that YOLOv8n–BackgroundSuppression is better than YOLOv10n.

These findings highlight the importance of employing a robust methodology when comparing models. Without such rigor, conclusions might be biased by factors such as the partitions used or the random variability inherent in the model training processes.

Furthermore, while the experimental validation in this paper focuses on the NEU dataset due to its relevance and widespread use in surface defect detection, the proposed benchmarking methodology is designed to be model and dataset-agnostic. Additionally, the methodology has been evaluated using the Workpieces Image Dataset (Riego del Castillo & Sánchez González, 2021), a dataset used for classifying surface wear on industrial components.

In particular, the statistical evaluation pipeline, partitioning strategies, and reproducibility guidelines are applicable to other small-scale industrial datasets. These datasets typically face similar challenges, such as limited sample sizes, class imbalance, and application-specific constraints. Thus, our framework can serve as a useful tool for rigorous evaluation across a wide range of real-world industrial contexts.

Conclusion

This paper analyzes the performance of different surface defect detection models in images, comparing their results using the NEU dataset, one of the most widely used in the industry. The reviewed studies present significantly varied results, with one key factor being the lack of consistency in the partitions used for training and testing the models. This makes direct comparisons between different approaches challenging.

To address this issue, a methodology based on stratified cross-validation and statistical significance analysis using ANOVA is proposed. The main objective of this methodology is to determine whether small improvements in results are statistically significant. Otherwise, such improvements may be the result of chance rather than genuine optimization of the model's architecture.

The results obtained in this study reveal that an improvement in the AP_{50} value, even by as much as 2.4%, may not be statistically significant. This highlights a critical point in the evaluation of deep learning models: relying solely on metrics such as AP_{50} without considering their statistical significance can lead to erroneous conclusions. This phenomenon can be attributed to several factors, including variability introduced by data partitioning, initial training conditions, or even random fluctuations inherent to the training processes.

An additional important aspect is the lack of public implementations in some studies, which leads to reproducibility issues due to the inability to access the source code. In many surface defect detection studies, the architecture used is described, but the corresponding code is rarely shared. This presents a significant barrier to replication and validation of results, as implementing these architectures becomes complex and difficult to reproduce without key details often missing from publications. In this context, the paper offers a constructive critique of the lack of open practices in the community, emphasizing the importance of researchers sharing their implementations.

Acknowledgements This work was supported by the Spanish National Plan for Research, Development and Innovation under Project PID2021-124383OB-I00 and the Asturian Science, Business Competitiveness and Innovation Agency (SEKUENS) under Project SEK-25-GRU-GIC-24-003 and FUO-319-24.

Author contributions Darío G. Lema: Methodology, software, validation, formal analysis, investigation, data curation, writing—original draft, writing—review and editing, visualization. Lidia Sánchez-González: Validation, investigation, writing—review and editing. Rubén Usamentiaga: Conceptualization, resources, supervision, project administration, funding acquisition, writing—review and editing. F.J. delaCalle: Software, validation, investigation.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Data availability The NEU dataset is publicly available at Kaggle.

Code availability The implementations of the models YOLOv8n–DefectDef, YOLOv8n–Unet, and YOLOv8n–BackgroundSuppression can be found in this repository.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Akhyar, F., Liu, Y., Hsu, C.-Y., Shih, T. K., & Lin, C.-Y. (2023). FDD: A deep learning-based steel defect detectors. *The International Journal of Advanced Manufacturing Technology, 126*(3–4), 1093–1107. https://doi.org/10.1007/s00170-023-11087-9

Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal speed and accuracy of object detection*. arXiv. https://doi.org/10.48550/arXiv.2004.10934

Castillo, V., & Sánchez González, L. (2021). Workpieces image dataset. *Zenodo*. Accessed 17 July 2025

Chen, W., Yang, K., Yu, Z., Nie, F., & Chen, C. L. P. (2025). Adaptive broad network with graph-fuzzy embedding for imbalanced noise data. *IEEE Transactions on Fuzzy Systems, 33*(6), 1949–1962. https://doi.org/10.1109/TFUZZ.2025.3543369

Chien, C.-T., Ju, R.-Y., Chou, K.-Y., Xieerke, E., & Chiang, J.-S. (2024). *YOLOv8-AM: YOLOv8 with attention mechanisms for pediatric wrist fracture detection*. arXiv. arXiv:2402.09329

Colosimo, B. M., Cavalli, S., & Grasso, M. (2020). A cost model for the economic evaluation of in-situ monitoring tools in metal additive manufacturing. *International Journal of Production Economics, 223*, 107532. https://doi.org/10.1016/j.ijpe.2019.107532

Dai, J., Qi, H., Xiong, Y., Li, Y., Zhang, G., Hu, H., & Wei, Y. (2017). Deformable convolutional networks. In *Proceedings of the IEEE international conference on computer vision* (pp. 764–773).

delaCalle, F. J., Fernández, A., Lema, D. G., Usamentiaga, R., & García, D. F. (2025). Automatic system for deformation measurement of anodes in an electrolytic process. *Measurement, 240*, 115626. https://doi.org/10.1016/j.measurement.2024.115626

Ding, Y., Zhao, Q., Li, T., Lu, C., Tao, L., & Ma, J. (2024). A rail defect detection framework under class-imbalanced conditions based on improved you only look once network. *Engineering Applications of Artificial Intelligence, 138*, 109351. https://doi.org/10.1016/j.engappai.2024.109351

He, Y., Song, K., Meng, Q., & Yan, Y. (2020). An end-to-end steel surface defect detection approach via fusing multiple hierarchical features. *IEEE Transactions on Instrumentation and Measurement, 69*(4), 1493–1504. https://doi.org/10.1109/TIM.2019.2915404

Jocher, G. (2020). *YOLOv5 by Ultralytics*. https://doi.org/10.5281/zenodo.3908559

Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11*. https://github.com/ultralytics/ultralytics

Kou, X., Liu, S., Cheng, K., & Qian, Y. (2021). Development of a YOLO-V3-based model for detecting defects on steel strip surface. *Measurement, 182*, 109454. https://doi.org/10.1016/j.measurement.2021.109454

Lema, D. G., Pedrayes, O. D., Usamentiaga, R., & García, D. F. (2023). Improved detection of subsurface defects through active thermography and ensembling techniques. *Quality Engineering, 35*(4), 669–685. https://doi.org/10.1080/08982112.2023.2177871

Lema, D. G., Usamentiaga, R., & García, D. F. (2024). Enhancing automated inspection in metal industries: Zero-shot segmentation of surface defects using bounding box prompts. *Measurement Science and Technology, 35*(8), 085604. https://doi.org/10.1088/1361-6501/ad48a4

Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., & Wei, X. (2022). *YOLOv6: A single-stage object detection framework for industrial applications*. arXiv

Li, Y. (2024). *Sop-net: An industrial-grade method for identifying small defect detection*. Available at SSRN 4910363

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), *Computer vision—ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48

Lin, Y., Yu, Z., Yang, K., & Philip Chen, C. (2025). Ensemble denoising autoencoders based on broad learning system for time-series anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*. https://doi.org/10.1109/TNNLS.2025.3548941

Liu, G., Chu, M., Gong, R., & Zheng, Z. (2025). Global attention module and cascade fusion network for steel surface defect detection.

Pattern Recognition, 158, 110979. https://doi.org/10.1016/j.patcog.2024.110979

Lobanova, I., Vodopyanov, A., & Kalinichenko, A. (2021). Investigation of the liquid flow on rough surfaces to solve the problems of liquid penetrant testing. In I. V. Minin, S. Uchaikin, A. Rogachev, & O. Starý (Eds.), *Progress in material science and engineering* (pp. 89–99). Springer. https://doi.org/10.1007/978-3-030-68103-6_9

Lv, Z., Li, Y., Qian, S., Wu, L., & Yang, Y. (2025). Union channel pruning-based u2net for online surface defect segmentation of aluminum strips in production processes. *Journal of Intelligent Manufacturing, 36*(3), 1579–1602. https://doi.org/10.1007/s10845-023-02317-8

Medak, D., Posilović, L., Subašić, M., Budimir, M., & Lončarić, S. (2022). DefectDet: A deep learning architecture for detection of defects with extreme aspect ratios in ultrasonic images. *Neurocomputing, 473*, 107–115. https://doi.org/10.1016/j.neucom.2021.12.008

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 779–788). Las Vegas: IEEE. https://doi.org/10.1109/CVPR.2016.91

Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. In *2017 IEEE conference on computer vision and pattern recognition (CVPR)* (pp. 6517–6525). Honolulu: IEEE. https://doi.org/10.1109/CVPR.2017.690

Redmon, J., & Farhadi, A. (2018). *YOLOv3: An incremental improvement*. arXiv

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In N. Navab, J. Hornegger, W. M. Wells, & A. F. Frangi (Eds.), *Medical image computing and computer-assisted intervention—MICCAI 2015* (pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

Song, C., Chen, J., Lu, Z., Li, F., & Liu, Y. (2023). Steel surface defect detection via deformable convolution and background suppression. *IEEE Transactions on Instrumentation and Measurement, 72*, 1–9. https://doi.org/10.1109/TIM.2023.3277989

Song, G., Song, K., & Yan, Y. (2020). Saliency detection for strip steel surface defects using multiple constraints and improved texture features. *Optics and Lasers in Engineering, 128*, 106000. https://doi.org/10.1016/j.optlaseng.2019.106000

Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science, 285*, 858–864. https://doi.org/10.1016/j.apsusc.2013.09.002

Ultralytics. (2023). *Brief summary of YOLOv8 model structure*. https://github.com/ultralytics/ultralytics/issues/189. Accessed 09 June 2025

Usamentiaga, R., Ibarra-Castanedo, C., Klein, M., Maldague, X., Peeters, J., & Sanchez-Beato, A. (2017). Nondestructive evaluation of carbon fiber bicycle frames using infrared thermography. *Sensors (Basel, Switzerland), 17*(11), 2679. https://doi.org/10.3390/s17112679

Usamentiaga, R., Lema, D. G., Pedrayes, O. D., & Garcia, D. F. (2021). Automated surface defect detection in metals: A comparative review of object detection and semantic segmentation using deep learning. In *2021 IEEE industry applications society annual meeting (IAS)* (pp. 1–8). https://doi.org/10.1109/IAS48185.2021.9677231

Usamentiaga, R., Lema, D. G., Pedrayes, O. D., & Garcia, D. F. (2022). Automated surface defect detection in metals: A comparative review of object detection and semantic segmentation using deep learning. *IEEE Transactions on Industry Applications, 58*(3), 4203–4213. https://doi.org/10.1109/TIA.2022.3151560

Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., & Ding, G. (2024a). *YOLOv10: Real-time end-to-end object detection*. arXiv.arXiv:2405.14458

Wang, C.-Y., Bochkovskiy, A., Liao, & H.-Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *2023 IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 7464–7475). Vancouver: IEEE. https://doi.org/10.1109/CVPR52729.2023.00721

Wang, C.-Y., Yeh, I.-H., & Liao, H.-Y.M. (2024b). *YOLOv9: Learning what you want to learn using programmable gradient information*. arXiv

Wang, H., Yang, X., Zhou, B., Shi, Z., Zhan, D., Huang, R., Lin, J., Wu, Z., & Long, D. (2023). Strip surface defect detection algorithm based on YOLOv5. *Materials, 16*(7), 2811. https://doi.org/10.3390/ma16072811

Wang, X., Xuan, Y., Huang, X., & Yan, Q. (2024). Yolo-afk: Advanced fine-grained object detection for complex solder joints defect. *IEEE Access, 12*, 166238–166252. https://doi.org/10.1109/ACCESS.2024.3495540

Xie, J., Safdar, M., Romascanu, A. M., Lu, Y., Ko, H., Yang, Z., & Zhao, Y. F. (2024). Towards reproducible machine learning-based process monitoring and quality prediction research for additive manufacturing. In *International design engineering technical conferences and computers and information in engineering conference, volume 2A: 44th Computers and Information in Engineering Conference (CIE)* (pp. 02–02033). https://doi.org/10.1115/DETC2024-144461

Xie, J., Sun, L., & Zhao, Y. F. (2025). On the data quality and imbalance in machine learning-based design and manufacturing a systematic review. *Engineering, 45*, 105–131. https://doi.org/10.1016/j.eng.2024.04.024

Yang, K., Chen, W., Shi, Y., Yu, Z., & Chen, C. L. P. (2024). Simplified kernel-based cost-sensitive broad learning system for imbalanced fault diagnosis. *IEEE Transactions on Artificial Intelligence, 5*(12), 6629–6644. https://doi.org/10.1109/TAI.2024.3478191

Zhao, C., Shu, X., Yan, X., Zuo, X., & Zhu, F. (2023). Rdd-yolo: A modified yolo for detection of steel surface defects. *Measurement, 214*, 112776. https://doi.org/10.1016/j.measurement.2023.112776

Zhu, X., Hu, H., Lin, S., & Dai, J. (2019). Deformable ConvNets V2: More deformable, better results. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9308–9316).

Zolfaghari, A., Zolfaghari, A., & Kolahan, F. (2018). Reliability and sensitivity of magnetic particle nondestructive testing in detecting the surface cracks of welded components. *Nondestructive Testing and Evaluation, 33*(3), 290–300. https://doi.org/10.1080/10589759.2018.1428322

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.